

Why this?

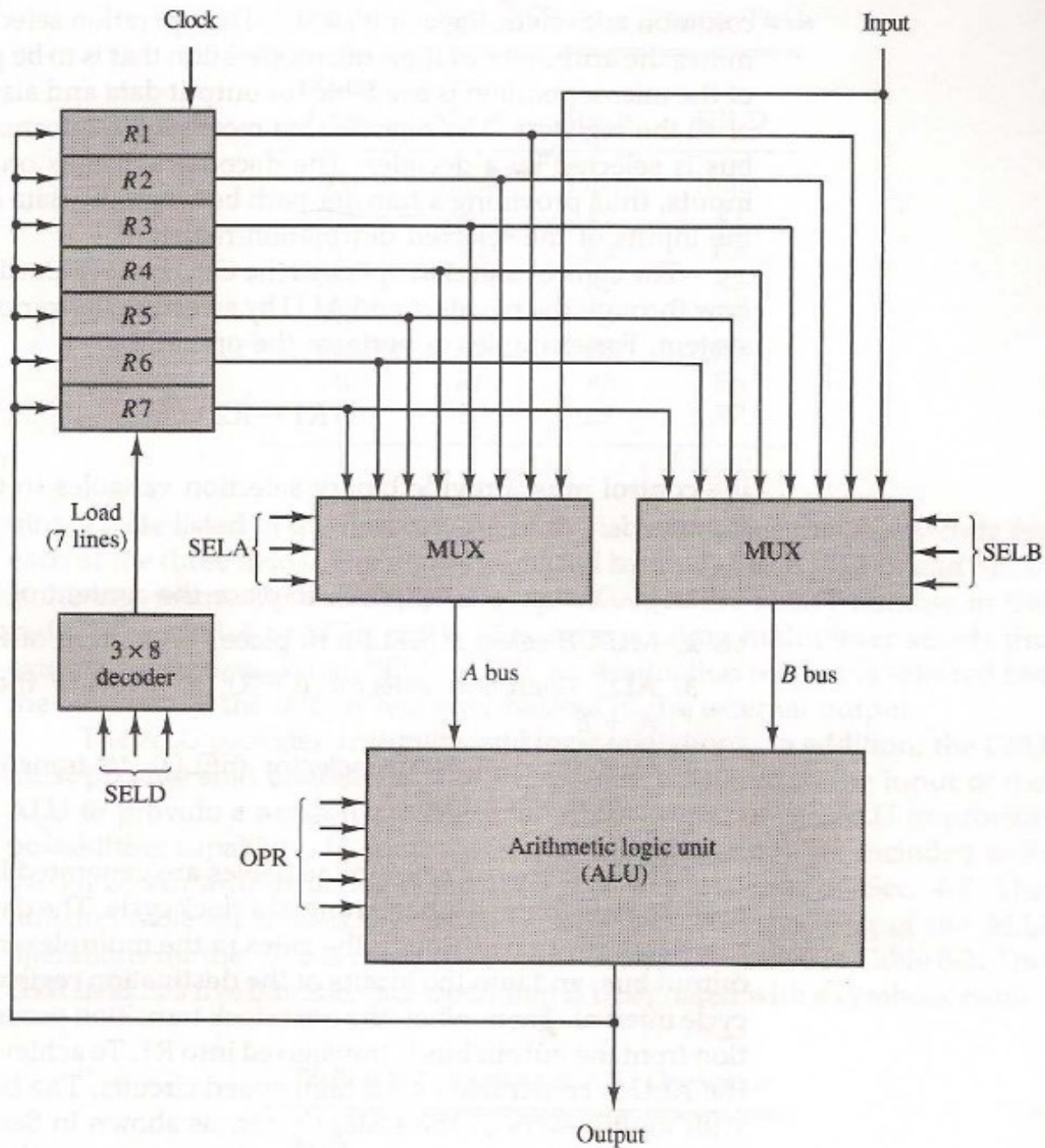
- ↳ Memory locations are needed for storing intermediate results
- ↳ Memory access is the most time-consuming operation in a computer.
- ↳ More convenient & efficient way is to use registers for storing intermediate results.

3 major parts of CPU:

1. ALU

2. Control Unit

3. Register Set



(a) Block diagram

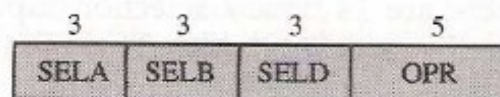


TABLE 8-2 Encoding of ALU Operations

OPR Select	Operation	Symbol
00000	Transfer A	TSFA
00001	Increment A	INCA
00010	Add $A + B$	ADD
00101	Subtract $A - B$	SUB
00110	Decrement A	DECA
01000	AND A and B	AND
01010	OR A and B	OR
01100	XOR A and B	XOR
01110	Complement A	COMA
10000	Shift right A	SHRA
11000	Shift left A	SHLA

TABLE 8-1 Encoding of Register Selection Fields

Binary Code	SELA	SELB	SELD
000	Input	Input	None
001	R1	R1	R1
010	R2	R2	R2
011	R3	R3	R3
100	R4	R4	R4
101	R5	R5	R5
110	R6	R6	R6
111	R7	R7	R7

Stack Organization:

↳ Two types of stack.

1. Register stack:

↳ Fixed size

↳ Makes use of flags to check if empty or full

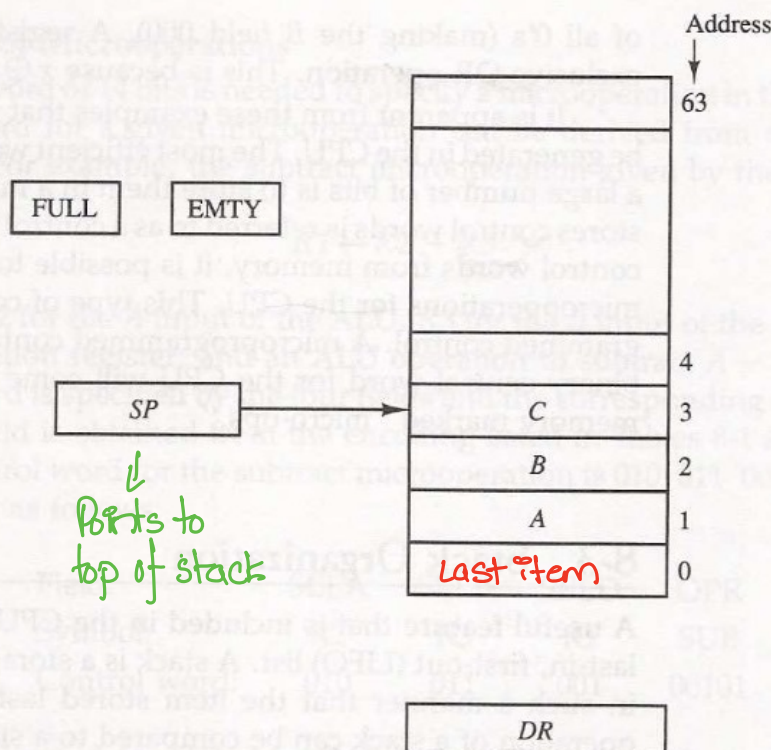


Figure 8-3 Block diagram of a 64-word stack.

PUSH:

$SP \leftarrow SP + 1$

$M[SP] \leftarrow DR$

If $(SP = 0)$ then $(FULL \leftarrow 1)$

$EMPTY \leftarrow 0$

POP:

$DR \leftarrow M[SP]$

$SP \leftarrow SP - 1$

If $(SP = 0)$ then $(EMPTY \leftarrow 1)$

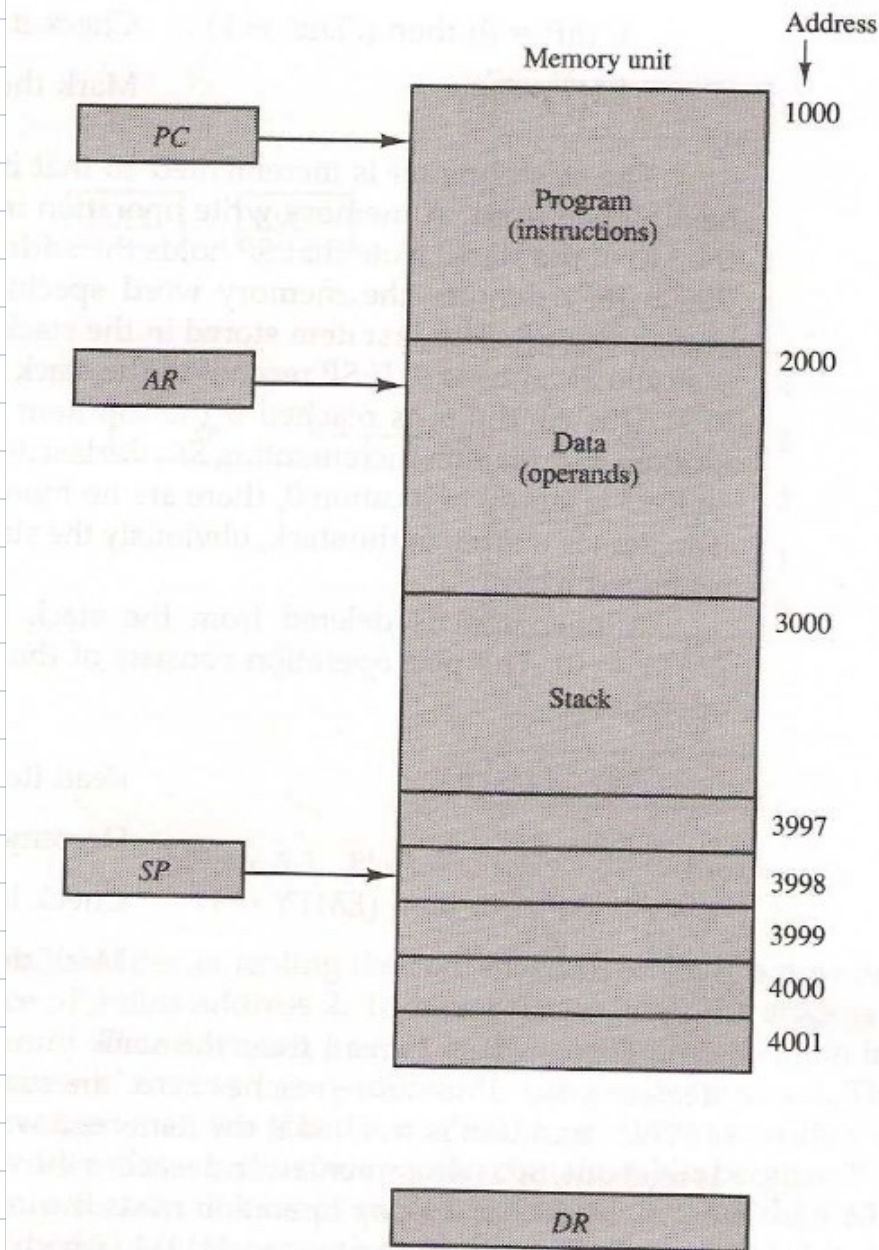
$FULL \leftarrow 0$

2. Memory Stack

- ↳ Dynamic size
- ↳ No need for flags

↳ Makes use of two registers to check for stack limits

- Upper-limit register → After PUSH, SP compared with it
- Lower-limit register → After POP, SP compared with it



PUSH:

$SP \leftarrow SP - 1$
 $M[SP] \leftarrow DR$

POP:

$DR \leftarrow M[SP]$
 $SP \leftarrow SP + 1$

Reverse Polish Notation

$A+B$ Infix notation

$+AB$ Prefix/Polish notation

AB+ Postfix/Reverse Polish Notation

STEPS to evaluate RPN:

1. Read from Left to Right
2. If operator, then remove 2 from stack, perform operation & push onto stack

$$(A+B) * [C * (D+E) + F]$$

$$AB+ CDE+ * F+$$

Instruction Formats:

1. Op-code
2. Address field → Specify memory address OR processor reg
3. Mode field → Specify the addressing mode

↳ Most computers fall into three types of CPU organization:

1. Single accumulator Organization

↳ Basic computer

↳ All ops are performed with implied AC.

↳ Uses one address field

$$\text{ADD } X \quad AC \leftarrow AC + M[X]$$

2. General Register type Organization

↳ Needs three register address fields

$$\text{ADD } R1, R2, R3 \quad R1 \leftarrow R1 + R2$$

$$\text{ADD } R1, R2 \quad R1 \leftarrow R1 + R2$$

↳ Computers with multiple processors registers use MOV for register transfer

MOV R1, R2 $R1 \leftarrow R2$

3. Stack Organized CPU

↳ No address

PUSH X Push $M[X]$

ADD pop two numbers from stack, add them & push onto stack

↳ Evaluating $X = (A+B) * (C+D)$ using zero, one, two & three address registers.

1. Three address instructions

ADD	R1, A, B	$R1 \leftarrow M[A] + M[B]$
ADD	R2, C, D	$R2 \leftarrow M[C] + M[D]$
MUL	X, R1, R2	$M[X] \leftarrow R1 * R2$

+ Short program

- Require too many bits to specify 3 address

2. Two - address instruction

MOV	R1, A	$R1 \leftarrow M[A]$
ADD	R1, B	$R1 \leftarrow R1 + M[B]$
MOV	R2, C	$R2 \leftarrow M[C]$
ADD	R2, D	$R2 \leftarrow R2 + M[D]$
MUL	R1, R2	$R1 \leftarrow R1 * R2$
MOV	X, R1	$M[X] \leftarrow R1$

3. One-address instruction

↳ Uses implied AC for all Data manipulation

LOAD	A	$AC \leftarrow M[A]$
ADD	B	$AC \leftarrow AC + M[B]$
STORE	T	$M[T] \leftarrow AC$
LOAD	C	$AC \leftarrow M[C]$
ADD	D	$AC \leftarrow AC + M[D]$
MUL	T	$AC \leftarrow AC * M[T]$
STORE	X	$M[X] \leftarrow AC$

4. Zero-address instruction

PUSH	A	$TOS \leftarrow A$
PUSH	B	$TOS \leftarrow B$
ADD		$TOS \leftarrow (A + B)$
PUSH	C	$TOS \leftarrow C$
PUSH	D	$TOS \leftarrow D$
ADD		$TOS \leftarrow (C + D)$
MUL		$TOS \leftarrow (C + D) * (A + B)$
POP	X	$M[X] \leftarrow TOS$

RISC instructions:

- ↳ Load & Store have one memory & one register address
- ↳ Computation type instructions have three processor register addresses.

LOAD	R1, A	$R1 \leftarrow M[A]$
LOAD	R2, B	$R2 \leftarrow M[B]$
LOAD	R3, C	$R3 \leftarrow M[C]$
LOAD	R4, D	$R4 \leftarrow M[D]$
ADD	R1, R1, R2	$R1 \leftarrow R1 + R2$
ADD	R3, R3, R2	$R3 \leftarrow R3 + R2$
MUL	R1, R1, R3	$R1 \leftarrow R1 * R3$
STORE	X, R1	$M[X] \leftarrow R1$

Addressing Modes

why?

- ↳ Provide versatility to the programmer
- ↳ To reduce no. of bits in addressing field of the instruction.

↳ Distinct Binary Code →

Opcode	Addressing mode field
--------	-----------------------

↳ Single binary code →

Op-code + Addressing Mode field

1. Implied Mode:

↳ Operands are specified implicitly in the definition of the instructions.

↳ For example,

Complement AC → Operand is in AC

↳ All register reference instruction that use an accumulator are implied mode.

↳ Zero-address instruction in stack-organized computer are implied mode.

2. Immediate Mode:

↳ Has operand field instead of address field
↳ Contains the actual operand

3. Register mode:

↳ When address field specifies a processor register

it's register mode.

- ↳ Operands are in registers within the CPU.

4. Register Indirect Mode:

- ↳ Instruction specifies a register in CPU whose contents give address of the operand in memory.

5. Auto-increment/Decrement:

↳ Similar to Indirect mode except:

- ↳ Register is incremented after it's value is used to access memory

- ↳ Register is decremented before it's value is used to access memory

6. Direct Address mode

↳ Effective address is equal to address part of instruction

7. Indirect Address mode:

↳ Address field of the instruction gives address of where effective address is stored in memory.

↳ A few addressing modes require that the address field of the instruction to be added to the content of a specific register in CPU.

$$\text{Effective address} = \text{address part (ADR) of instruction} + \text{content of CPU register.}$$

8. Relative addressing mode:

$$\text{Effective address} = \text{ADR} + \text{PC}$$

↳ 2's complement number

↳ When ADR is added to PC, the result produces an EA whose position in memory is relative to the address of the next instruction. ↵

- ↳ Used in Branch-type instructions when branch address is in the area surrounding the instruction word itself.

↳ helps save bits.

9. Indexed Addressing Mode:

$$\text{Effective address} = \text{ADR} + \text{Index Register}$$

↳ special CPU registers that contains an index value.

↳ used for array accessing

Memory address: 1000 1004 1008

Value Stored : 42 17 93

$$\text{ADR} = 1000$$

$$\text{IR} = 0$$

$$\text{IR} = 4$$

$$\begin{aligned} \text{EA} &= \text{ADR} + \text{IR} \\ &= 1000 + 0 \\ &= 1000 \end{aligned}$$

$$\begin{aligned} \text{EA} &= \text{ADR} + \text{IR} \\ &= 1000 + 4 \\ &= 10004 \end{aligned}$$

$$M[\text{EA}] = 42$$

$$M[\text{EA}] = 17$$

↳ Index Registers holds the displacement from address part of instruction.

10. Base Register Addressing mode:

$$\text{Effective address} = \text{ADR} + \text{Base register}$$

↳ Base registers hold base address & address field of instruction gives the displacement.

Addressing Mode	Effective Address	Content of AC
Immediate Address Mode	201	500
Direct Address Mode	500	800
Indirect Address Mode	800	300
Register Mode		400
Register Indirect Mode	400	700
Relative Address Mode	702	325
Indexed Address Mode	600	900
Autoincrement Mode	400	700
Autodecrement Mode	399	450

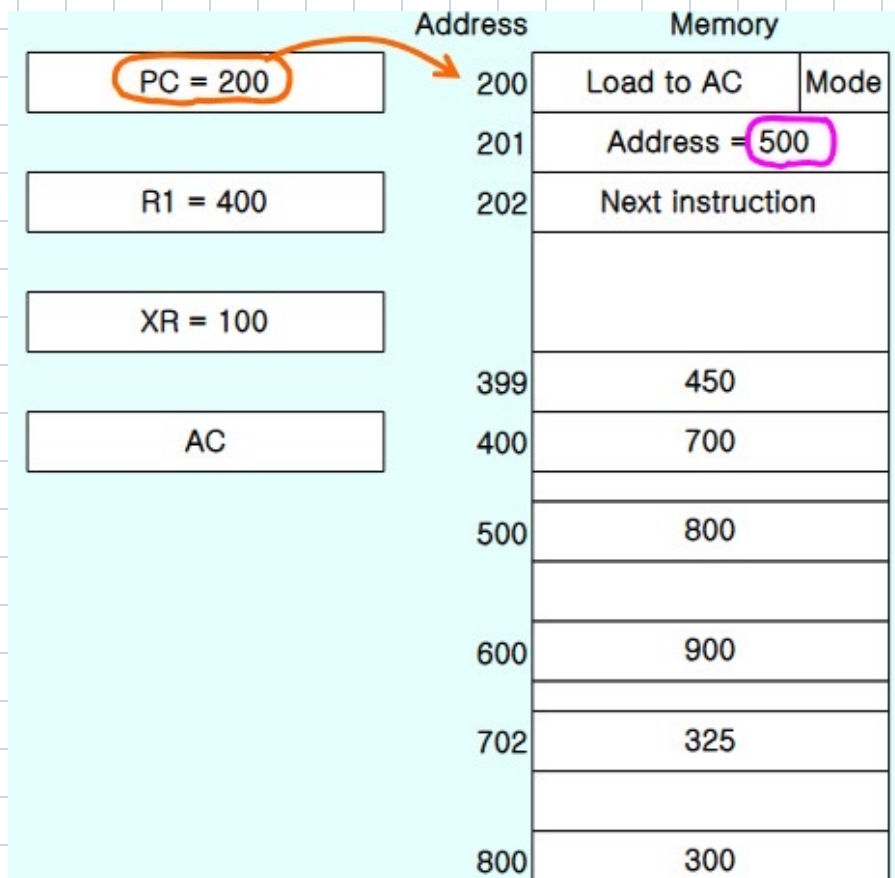
$$R1 = 400 + 1 \text{ (after)}$$

$$R1 = 400 - 1 \text{ (prior)}$$

$$R1 = 400$$

$$500 + 202 \text{ (PC)}$$

$$500 + 100 \text{ (XR)}$$



Data Transfer And Manipulation

↳ Most computer instructions can be classified as:

1. Data Transfer

• Typical Data Transfer Instruction : **Tab. 8-5**

- **Load** : transfer from memory to a processor register, usually an AC (*memory read*)
- **Store** : transfer from a processor register into memory (*memory write*)
- **Move** : transfer from one register to another register
- **Exchange** : swap information between two registers or a register and a memory word
- **Input/Output** : transfer data among processor registers and input/output device
- **Push/Pop** : transfer data between processor registers and a memory stack

• 8 Addressing Mode for the LOAD Instruction :

- **@** : Indirect Address
- **\$** : Address relative to PC
- **#** : Immediate Mode
- **()** : Index Mode, Register Indirect, Autoincrement register

TABLE 8-6 Eight Addressing Modes for the Load Instruction

Mode	Assembly Convention	Register Transfer
Direct address	LD ADR	$AC \leftarrow M[ADR]$
Indirect address	LD @ADR	$AC \leftarrow M[M[ADR]]$
Relative address	LD \$ADR	$AC \leftarrow M[PC + ADR]$
Immediate operand	LD #NBR	$AC \leftarrow NBR$
Index addressing	LD ADR(X)	$AC \leftarrow M[ADR + XR]$
Register	LD R1	$AC \leftarrow R1$
Register indirect	LD (R1)	$AC \leftarrow M[R1]$
Autoincrement	LD (R1)+	$AC \leftarrow M[R1], R1 \leftarrow R1 + 1$

TABLE 8-5 Typical Data Transfer Instructions

Name	Mnemonic
Load	LD
Store	ST
Move	MOV
Exchange	XCH
Input	IN
Output	OUT
Push	PUSH
Pop	POP

2. Data Manipulation:

- Arithmetic Instruction
- Logical & bit manipulation
- Shift Instruction

TABLE 8-7 Typical Arithmetic Instructions

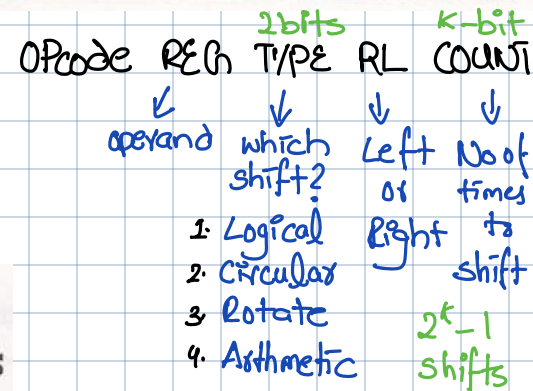
Name	Mnemonic
Increment	INC
Decrement	DEC
Add	ADD
Subtract	SUB
Multiply	MUL
Divide	DIV
Add with carry	ADDC
Subtract with borrow	SUBB
Negate (2's complement)	NEG

TABLE 8-8 Typical Logical and Bit Manipulation Instructions

Name	Mnemonic
Clear	CLR
Complement	COM
AND	AND
OR	OR
Exclusive-OR	XOR
Clear carry	CLRC
Set carry	SETC
Complement carry	COMC
Enable interrupt	EI
Disable interrupt	DI

TABLE 8-9 Typical Shift Instructions

Name	Mnemonic
Logical shift right	SHR
Logical shift left	SHL
Arithmetic shift right	SHRA
Arithmetic shift left	SHLA
Rotate right	ROR
Rotate left	ROL
Rotate right through carry	RORC
Rotate left through carry	ROLC



↳ Mnemonics that specify different data types:

ADDI	Add two binary integer numbers
ADDF	Add two floating-point numbers
ADDD	Add two decimal numbers in BCD

Program Control

↳ Branch = Jump instructions

↳ Status/Code bits

↳ Set/cleared as a result of an operation in the ALU

C (carry) set to 1 if C_8 is 1

S (sign) Set to 1 if F_7 is 1

Z (zero) Set to 1 if output of ALU is 0

V (overflow) $= C_8 \oplus C_7$

Example,

$$A - B = A + \bar{B} + 1$$

$$\begin{array}{r}
 C_7 \\
 1 \quad 1 \\
 1 \quad 1 \quad 1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \\
 1 \quad 1 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0 \\
 \hline
 1 \quad 1 \quad 1 \quad 0 \quad 1 \quad 1 \quad 1 \quad 0 \quad 0 \\
 C_8 \quad F_7 \quad F_6 \quad F_5 \quad F_4 \quad F_3 \quad F_2 \quad F_1 \quad F_0
 \end{array}$$

$$C = 1$$

$$S = 1$$

$$Z = 0$$

$$V = 0$$

$$\begin{array}{l}
 A = 11110000 \\
 B = 11101100
 \end{array}$$

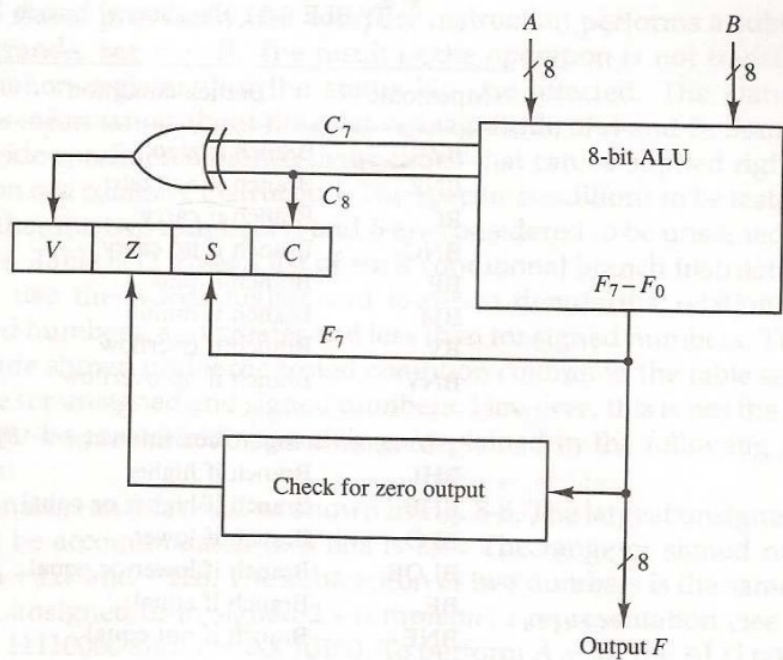


Figure 8-8 Status register bits.

TABLE 8-11 Conditional Branch Instructions

Mnemonic	Branch condition	Tested condition
BZ	Branch if zero	$Z = 1$
BNZ	Branch if not zero	$Z = 0$
BC	Branch if carry	$C = 1$
BNC	Branch if no carry	$C = 0$
BP	Branch if plus	$S = 0$
BM	Branch if minus	$S = 1$
BV	Branch if overflow	$V = 1$
BNV	Branch if no overflow	$V = 0$

Unsigned compare conditions ($A - B$)

BHI	Branch if higher	$A > B$
BHE	Branch if higher or equal	$A \geq B$
BLO	Branch if lower	$A < B$
BLOE	Branch if lower or equal	$A \leq B$
BE	Branch if equal	$A = B$
BNE	Branch if not equal	$A \neq B$

Signed compare conditions ($A - B$)

BGT	Branch if greater than	$A > B$
BGE	Branch if greater or equal	$A \geq B$
BLT	Branch if less than	$A < B$
BLE	Branch if less or equal	$A \leq B$
BE	Branch if equal	$A = B$
BNE	Branch if not equal	$A \neq B$

TABLE 8-10 Typical Program Control Instructions

Name	Mnemonic
Branch	BR
Jump	JMP
Skip	SKP
Call	CALL
Return	RET
Compare (by subtraction)	CMP
Test (by ANDing)	TST

Sub-routine Call & Return

↳ After branch to sub-routine, to return you need to store the return address.

1. Address of next instruction stored in temporary storage
2. PC moved to start of sub-routine
3. Transfer return address from temporary location to PC

$SP \leftarrow SP - 1$
 $M[SP] \leftarrow PC$ } Call

$PC \leftarrow EA$

$PC \leftarrow M[SP]$
 $SP \leftarrow SP + 1$ } return

Program Interrupt

↳ Transfer program control from currently program to another service as a result of an external or internal generated request

↳ Control returns to the original program after the service program is executed.

Interrupt Service Program

- ↳ Initiated by an internal or external signal
- ↳ Address of ISP determined by hardware.

↳ Stores all information necessary to define the state of CPU

Sub-routine Call

- ↳ Initiated from execution of an instruction
- ↳ Address of sub-routine call is determined from address field of an instruction.

↳ Stores only PC

Program status Word

↳ Collection of all status bit conditions

Two CPU Operating Modes

- Supervisor (System) Mode: Privileged Instruction

↳ When the CPU is executing a program that is part of the operating system.

- User Mode:

↳ When the CPU is executing an user program

- 1) External Interrupts

- come from I/O device, from a timing device, from a circuit monitoring the power supply, or from any other external source

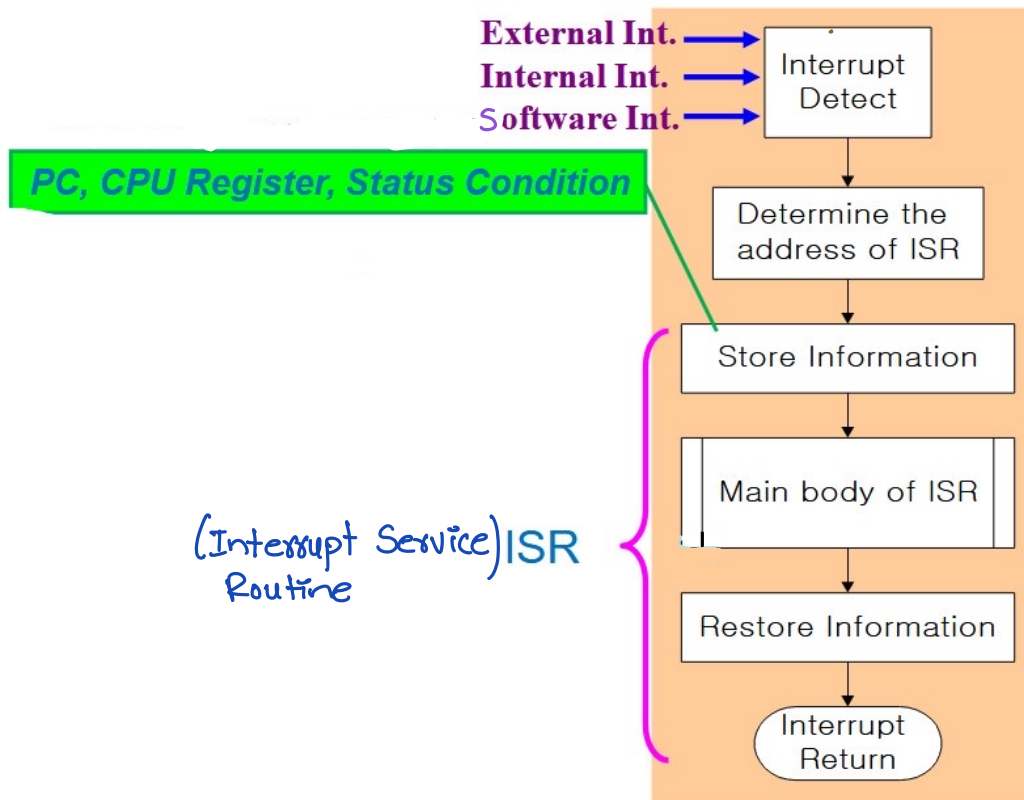
ISR

- 2) Internal Interrupts or TRAP

- caused by register overflow, attempt to divide by zero, an invalid operation code, stack overflow, and protection violation

- 3) Software Interrupts

- initiated by executing an instruction
- used by the programmer to initiate an interrupt procedure at any desired point in the program



8-8 Reduced Instruction Set Computer (RISC)

– Complex Instruction Set Computer (CISC)

• Major characteristics of a CISC architecture

- 1) A large number of instructions - typically from 100 to 250 instruction
- 2) Some instructions that perform specialized tasks and are used infrequently
- 3) A large variety of addressing modes - typically from 5 to 20 different modes
- 4) Variable-length instruction formats
- 5) Instructions that manipulate operands in memory

– Reduced Instruction Set Computer (RISC)

• Major characteristics of a RISC architecture

- 1) Relatively few instructions
- 2) Relatively few addressing modes
- 3) Memory access limited to load and store instruction
- 4) All operations done within the registers of the CPU
- 5) Fixed-length, easily decoded instruction format
- 6) Single-cycle instruction execution
- 7) Hardwired rather than microprogrammed control
- 8) A relatively large number of registers in the processor unit
- 9) Use of overlapped register windows to speed-up procedure call and return
- 10) Efficient instruction pipeline
- 11) Compiler support for efficient translation of high-level language programs into machine language programs

Saving & restoring registers
Passing of params & results

Avoids need for saving & restoring
register values

Overlapped Register Window

$$\text{Window size} = L + 2C + G$$

$$\text{No. of processor registers} = (L + C)W + G$$

G Global registers
L Local registers
C Common registers
W No. of windows

Example,

Proc A calls Proc B.

↳ $R_{26} - R_{31}$ for storing results/params of Proc B.

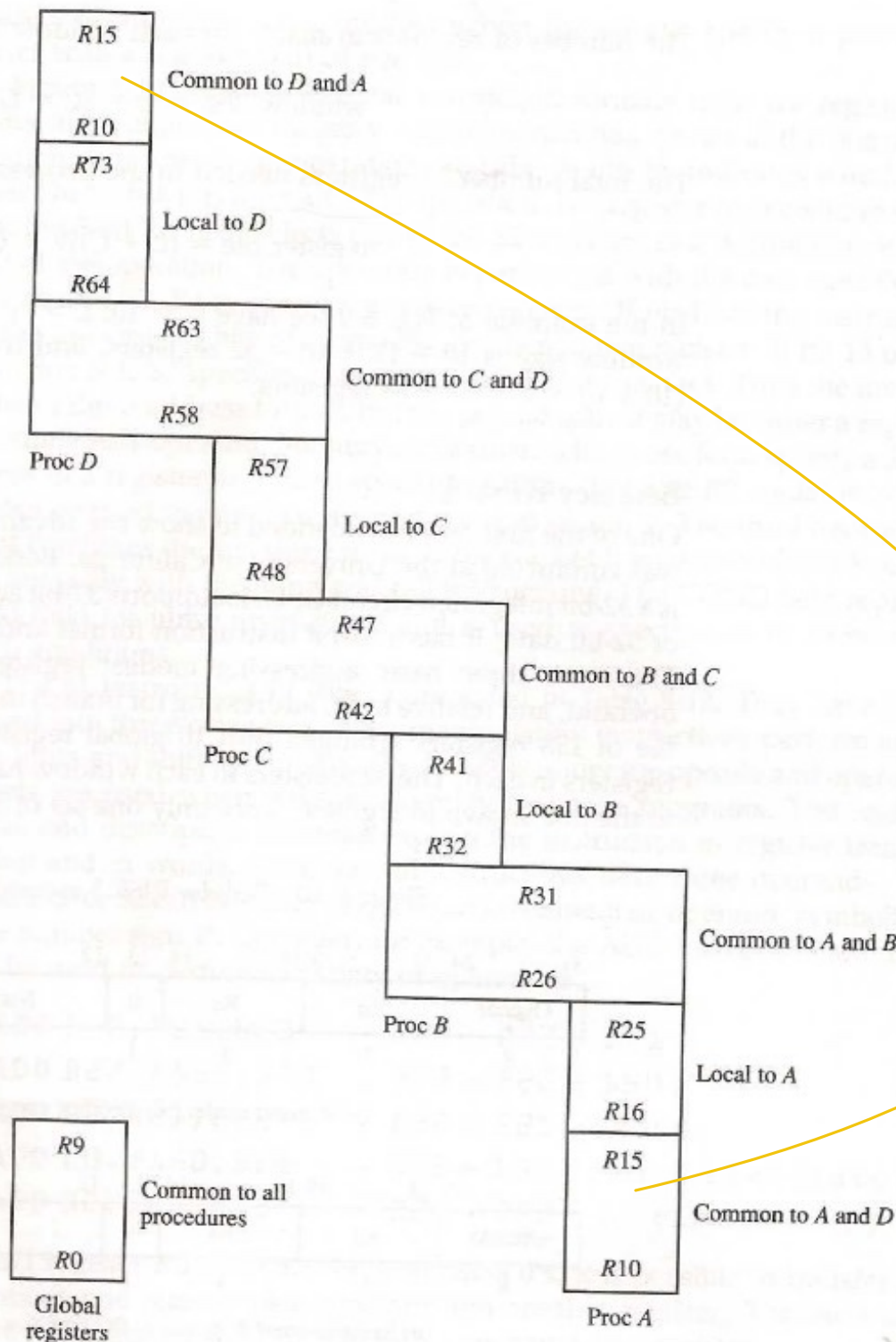


Figure 8-9 Overlapped register windows.

Book Qs:

Q1.

(a) 32×16

(b) $2^4 = 16$, so 4 select lines

(c) 4×16

(d) $32 + 32 + \overset{\text{carry in}}{1} = 65$

$32 + 1 = 33$

$\overset{\text{carry out}}{\downarrow}$

(e) $\overset{4}{\text{sel A}} \quad \overset{4}{\text{sel B}} \quad \overset{4}{\text{sel D}} \quad \overset{6}{\text{OPR}}$

Q3.

sel A sel B sel D

(a) 011 001 010 00010

(b) 100 — 100 00000

(c) 101 — 101 00110

(d) 001 — 110 11000

(e) 000 — 111 00000

Q2.

$30 + 80 + 10 = 120$

(Decoder signals propagate at the same time as Mux)

Q4

(a)

	R1	R2	R3	SUB	
a.	00101001	1100101			
	Input	Input	None	TSFA	
b.	00000000	00000000			Out ← Input
	R2	R2	R2	XOR	
c.	01001001	1001100			$R2 \leftarrow R2 \oplus R2$
	Input	R1	None	ADD	
d.	00000100	00000010			Out ← Input + R1
	R7	R5	R3	SHRA	
e.	11110001	1110000			$R3 \leftarrow \text{shl } R7$

Q5.

(a) $SP = 000000$

$2^6 = 64$

(b) Empty/0

Q6. PUSH:

$M[SP] \leftarrow DR$

$SP \leftarrow SP - 1$

POP:

$DR \leftarrow M[SP]$

$SP \leftarrow SP + 1$

Q7.

(a) $AB * CD * EF * + +$

(d) $ABCDE + * + * FGH + * /$

(b) $AB * A BD * CE * + * +$

(c) $ABCD * EFGH + * + * +$

Q8

(a) $ABCDE + * - /$

$$\frac{A}{(B - (C * (D + E)))}$$

(b) $ABCDE * / - +$

$$A + B - \frac{C}{D * E}$$

(c) $ABC * / D - EF / +$

$$\frac{A}{B * C} - D + \frac{E}{F}$$

(d) $ABCDEFGH + * * * + *$

$$A * (B + C * (D + E * (F + H)))$$

Q9.

$$3H + 1026 + * 8 + * = 616$$

Q10. $WC \leftarrow$ Address to Write
 $RC \leftarrow$ Address to Read
 $ASC \leftarrow$ No. of words

PUSH:

$M[WC] \leftarrow DR$
 $ASC \leftarrow ASC + 1$
 $WC \leftarrow WC + 1$
 if $(ASC = \text{all 1s})$
 $FULL \leftarrow 1$
 $Empty \leftarrow 0$

POP:

$DR \leftarrow M[RC]$
 $ASC \leftarrow ASC - 1$
 $RC \leftarrow RC + 1$
 if $(ASC = \text{all 0s})$
 $Empty \leftarrow 1$

Q11. $32 - 24 = 6$ 12 12
 $op - wde$ $ADR1$ $ADR2$

$2^6 = 256$
 but $256 - 250 = 6$
 $\underbrace{2 \text{ adr}}_{\text{instructions}} \quad \underbrace{6 \text{ left}}$

So, $6 \times 2^{12} = 24,576$

Q12.

$$AB - CDE * F - * + G H K * + /$$

(a) $SUB \ R1, A, B$ $R1 \leftarrow A - B$
 $MUL \ R2, D, E$ $R2 \leftarrow D * E$
 $SUB \ R2, R2, F$ $R2 \leftarrow R2 - F \ (D * E - F)$
 $MUL \ R2, R2, C$ $R2 \leftarrow R2 * C \ (D * E - F) * C$
 $ADD \ R3, R2, R1$ $R3 \leftarrow R1 + R2 \ A - B + (D * E - F) * C$
 $MUL \ R2, H, K$ $R2 \leftarrow H * K$
 $ADD \ R2, R2, G$ $R2 \leftarrow R2 + G \ (H + H * K)$
 $DIV \ R4, R3, R2$ $R4 \leftarrow R3 / R2$

(b)

MOV R1, A	$R1 \leftarrow A$	
SUB R1, B	$R1 \leftarrow R1 - B$	$(A - B)$
MOV R2, D	$R2 \leftarrow D$	
MUL R2, E	$R2 \leftarrow D * E$	
SUB R2, F	$R2 \leftarrow R2 - F$	$(D * E - F)$
MUL R2, C	$R2 \leftarrow R2 * C$	$C * (D * E - F)$
ADD R1, R2	$R1 \leftarrow R1 + R2$	$A - B + C * (D * E - F)$
MOV R3, H	$R3 \leftarrow H$	
MUL R3, K	$R3 \leftarrow R3 * K$	$H * K$
ADD R3, G	$R3 \leftarrow R3 + G$	$H + H * K$
DIV R1, R3	$R1 \leftarrow R1 / R3$	

$AB - CDE * F - * + HAK * + /$

(c)

LOAD A	$AC \leftarrow A$
SUB B	$AC \leftarrow AC - B$
STORE TMP	
LOAD D	$AC \leftarrow D$
MUL E	$AC \leftarrow D * E$
SUB F	$AC \leftarrow AC - F$
MUL C	$AC \leftarrow AC * C$
ADD TMP	$AC \leftarrow AC + TMP$
STORE TMP	
LOAD H	$AC \leftarrow H$
MUL K	$AC \leftarrow AC * K$
ADD G	$AC \leftarrow AC + G$
STORE WD	
LOAD TMP	
DIV WD	$AC \leftarrow AC / WD$

(d)

PUSH A
PUSH B
SUB
PUSH C
PUSH D
PUSH E
MUL
PUSH F
SUB
MUL
ADD
PUSH G
PUSH H
PUSH K
MUL
ADD
DIV

Q13.

5	3	6	18
op-code	mode	reg-ADR	ADR

Q14.

$W \rightarrow \text{Instruction}$
 $W+1 \rightarrow Y(\text{ADR})$

(a) $Y = Z$

$IR = X$

(b) $Z = M[Y]$

(c) $Z = W + 2 + Y$

$Z \rightarrow \text{operand}$

(d) $Z = X + Y$

750 $\rightarrow$ BRC (?)

$$= 500 - 751 = -251$$

500 → Hey

(a) Fetch inst
Fetch EA
FETCH OPR

(b) FETCH inst
FETCH EA & Transfer to PC

$$\Sigma A \text{ of OPL} = IR + AOR$$

$\mathcal{EA} = \mathbb{IR}$ } register indexed

Q20.
$$\begin{array}{cccccccc} 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ \hline \end{array}$$

AND	1	0	0	0	1	0	0	0
OR	1	0	1	1	1	1	1	0
XOR	0	0	1	1	0	1	1	0

(a) EA = 400 300 inst
 301 AD R = 400

Q21.

(b) $EA = 301$

(C) $EA = 302 + 400$
 $= 702$

$$P_1 = 200$$

(d) $EA = 200$

(e) $EA = 200 + 400$
 $= 600$

(a) AND 0011
(b) OR 0011
(c) XOR 0110

[illegible]

Q22.

1. 00111101
2. 11110110
3. 00111101
4. 11110110 *overflow*
5. 10111101
6. 11110110
7. 10111101, C=1
8. 11110111, C=0

01111011, C=1

LSB=1
MSB=0

LSB=1
C=1

MSB=0
C=0

Q23.

$$83 = 01010011$$

$$-83 = 10101101$$

$$68 = 01000100$$

$$-68 = 10111100$$

10101100
1

(a)
$$\begin{array}{r} 10101101 \\ 01000100 \\ \hline 11110001 \end{array}$$
 (b)
$$\begin{array}{r} 10111100 \\ 10101101 \\ \hline 00110001 \end{array}$$
 (c)
$$11011110$$

$$11110001$$

$$00110001$$
 (d)
$$01011010$$
 (overflow)

Q24.

$$\overline{F_0} \overline{F_1} \overline{F_2} \dots \overline{F_3} = \overline{(F_0 + F_1 + \dots + F_3)}$$

$$R = 72$$

Q25.

(a)
$$\begin{array}{r} C6 \\ 72 \\ \hline 11000110 \\ 01110011 \\ \hline 100111001 \end{array}$$

$$C=1 \quad S=0 \quad Z=0 \quad V=0$$

(b)
$$\begin{array}{r} 00011110 \\ 01110011 \\ \hline 10010001 \end{array}$$

$$S=1, C=0, Z=0, V=1$$

Q26.

unsigned

Q27.

Signed

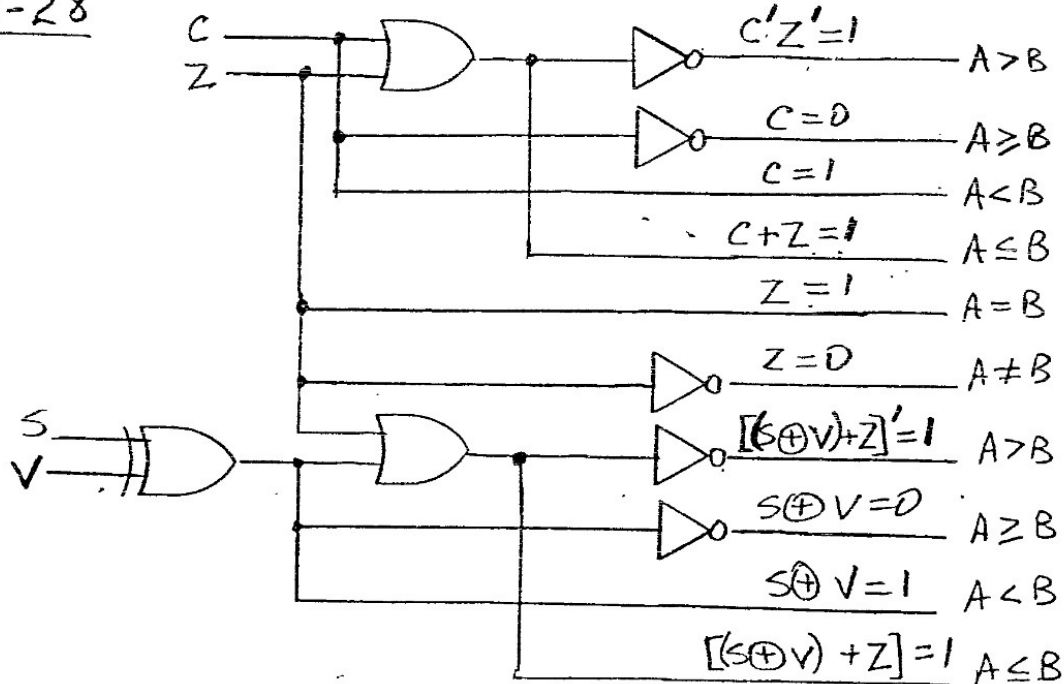
Relation	Condition of Status Bits
$A > B$	$C = 0$ and $Z = 0$
$A \geq B$	$C = 0$
$A < B$	$C = 1$
$A \leq B$	$C = 1$ or $Z = 1$
$A = B$	$Z = 1$
$A \neq B$	$Z = 0$

Table for Problem 8-27

Relation	Condition of Status Bits
$A > B$	$(S \oplus V) = 0$ and $Z = 0$
$A \geq B$	$(S \oplus V) = 0$
$A < B$	$(S \oplus V) = 1$
$A \leq B$	$(S \oplus V) = 1$ or $Z = 1$
$A = B$	$Z = 1$
$A \neq B$	$Z = 0$

Q28.

8-28



Q29.

-128 64 32 16 8 4 2 1

(a) $A = 01000001$

(1) 65

(2) 65

-128 64 32 16 8 4 2 1

 $B = 10000100$

(1) 132

(2) -124

(b) 01000001 (c) $C=0 \quad V=0 \quad Z=0 \quad S=1$

1 0 0 0 0 1 0 0

(d) BNZ BNC BM BNV

1 0 0 0 0 1 0 1
-128 64 32 16 8 4 2 1

(1) 133

(2) -123

Q30. C_7

(a) $\begin{array}{r} 01000001 \\ 01111100 \\ \hline \end{array}$ $\begin{array}{l} 65 \\ 132 \end{array}$

C_8 1 0 1 1 1 1 0 1 $-67/189$

$\overline{B}+1 = \begin{array}{r} 01111011 \\ 01111100 \end{array}$

(b) $C=0$ $Z=0$

$\begin{array}{r} 01100100 \\ + 10111101 \end{array}$

(c) BLO BNE BLOE

1 0 0 1 0 0 0 0 1
 $C_8 = 1$

Q31.

(a) $\begin{array}{r} 01000001 \\ 01111100 \\ \hline \end{array}$ $\begin{array}{l} 65 \\ -124 \end{array}$
0 1 1 1 1 1 0 1

Q32.

SP (3560) $\rightarrow$ 5320

(a) PC = 1120 SP = 3560 TOS = 5320

(b) PC = 6720 SP = 3559 TOS = 1222

(c) PC = 1222 SP = 3560 TOS = 5320

Q35.

1120 $\rightarrow$ sub-routine
1121 $\rightarrow$ ADR = 6720

(a) $SP \leftarrow SP - 1$
 $M[SP] \leftarrow PC$
 $SP \leftarrow SP - 1$
 $M[SP] \leftarrow PSW$
 $TR \leftarrow IAD$
 $PSW \leftarrow M[TR]$
 $TR \leftarrow TR + 1$

(b) $PSW \leftarrow M[SP]$
 $SP \leftarrow SP + 1$
 $PC \leftarrow M[SP]$
 $SP \leftarrow SP + 1$

PSW
PC

Q34.

External

Internal

$$PC \leftarrow M[TR]$$

Q37.

$$\text{Window Size} = h + 2C + L$$

$$\text{No. of Reg} = (L + C) \times W + G$$

- ↳ Input
- ↳ Output
- ↳ Time up
- ↳ Device Error
- ↳ Disk Ready

- ↳ Div by Zero
- ↳ Seg fault
- ↳ Overflow
- ↳ illegal inst
- ↳ Access violation

Comp 1

$$\begin{aligned} WS &= 10 + 2(6) + 10 \\ &= 32 \end{aligned}$$

$$\begin{aligned} \text{NOR} &= (10 + 6)8 + 10 \\ &= 138 \end{aligned}$$

Comp 2

$$\begin{aligned} WS &= 8 + 2(8) + 8 \\ &= 32 \end{aligned}$$

$$\begin{aligned} \text{NOR} &= (8 + 8)4 + 8 \\ &= 72 \end{aligned}$$

Comp 3

$$\begin{aligned} WS &= 16 + 2(16) + 16 \\ &= 64 \end{aligned}$$

$$\begin{aligned} \text{NOR} &= (16 + 16)16 + 16 \\ &= 528 \end{aligned}$$

Book Qs.

OLD PRAC

Q.7.

(a) $AB * CO * EF * ++$

(b) $AB * ABD * CE * + * +$

(C) $ABCD * EFGH + * + * +$

(d) $ABCDE + * + * FGHI + * /$

Q8.

(a)
$$\frac{A}{B - (D + E) * C}$$

$$(b) A \neq B - \frac{C}{(D \neq E)}$$

$$(C) \frac{A}{(B+C)} - D + \frac{E}{F}$$

(d) $A * B + C * D + E * (F + G)$

Q19.

[illegible]

1	0	0	0	0	0	1	0	0	0	0	1	1	0	0	0	1	1	0	0	0	0	1	0	0	0	0	0	1	0	0	1				
8								2				1				8				C				2				0				9			

$$1 + 1 * (1 + 1 + 1 * (1 + 1))$$

Handwritten diagram illustrating the addition of 1 and 2 to get 3, and then 1 and 3 to get 4. The numbers are written in a sequence: 1, 2, 3, 4. Wavy lines connect the numbers to show the progression: 1 + 2 = 3, and 1 + 3 = 4.

Q21.

Q22. $C = 1$

- (a) And with 0011 01111011
- (b) OR with 1100 1. 00111101
- (c) XOR with 0110 2. 11110110

Bit Rotation Operations - Step by Step

WITHOUT CARRY:

Left Rotation (ROL) of 10110111:

Original value: 10110111

1. Leftmost bit (MSB) is 1
2. Shift all bits left one position: 01101110
3. Place captured MSB (1) in rightmost position $C = 1$

Result: 01101111

MSB=0

LSB=1

$C = 0$

5. 11110110

6. 10111101

7. 10111101

8. 11110111

(overflow)

Right Rotation (ROR) of 10110111:

Original value: 10110111

1. Rightmost bit (LSB) is 1
2. Shift all bits right one position: 01011011
3. Place captured LSB (1) in leftmost position

Result: 11011011

MSB $\rightarrow$ MS

WITH CARRY:

Left Rotation through Carry (RCL) of 10110111 with $C=0$:

Original value: 10110111, $C=0$

1. Leftmost bit (MSB) is 1
2. This 1 goes to the carry flag: $C=1$
3. Shift all bits left one position: 01101110
4. Original carry value (0) goes to rightmost position

Result: 01101110, $C=1$

Right Rotation through Carry (RCR) of 10110111 with $C=1$:

Original value: 10110111, $C=1$

1. Rightmost bit (LSB) is 1
2. This 1 goes to the carry flag: $C=1$
3. Shift all bits right one position: 01011011
4. Original carry value (1) goes to leftmost position

Result: 11011011, $C=1$

Q23.

$$SP = 3560$$

(a) $PC = 1222, SP = 3660,$

$$SP \rightarrow 5320$$

(b) $PC = 6720, SP = 3659, 1222 \leftarrow SP$
 5320

(c) $PC = 1222, SP = 3650 \quad 5320 \leftarrow SP$

$$\begin{array}{l} 1120 \\ 1221 \end{array} \quad \begin{array}{l} \text{Call} \\ 6720 \text{ (ADR)} \end{array}$$